

CX-Aicode 插件 AI 编程标准作业流程 SOP（修订版）

ChatGPT

- CX-Aicode 插件 AI 编程标准作业流程 SOP（修订版）
 - 0. 文档定位
 - 0.1 本次修订说明
- 1. 插件架构与 SOP 设计依据
 - 1.1 插件的核心设计思想
 - 1.2 标准主流程
 - 1.3 功能边界与可执行性原则
 - 1.4 插件目录与产物目录
- 2. 角色与职责
 - 2.1 角色定义
 - 2.2 职责分离原则
- 3. 环境准备与安装 SOP
 - 3.1 前置环境
 - 3.2 插件安装
 - 3.3 业务项目初始化
 - 3.4 初始化方式选择
- 4. 状态文件与门禁规则
 - 4.1 `.cx/status/index.json` 的作用
 - 4.2 CodeReview 状态规则
 - 4.3 Agent 输出契约
 - 4.4 `.cx/work/` 三文件机制
 - 4.5 Hook 门禁机制
- 5. SOP A：新需求 / 新功能开发
 - 5.1 适用场景
 - 5.2 输入物
 - 5.3 操作步骤
- 6. SOP B：已有项目增量变更 / Brownfield 修改
 - 6.1 适用场景
 - 6.2 推荐流程
 - 6.3 操作步骤
- 7. SOP C：Bug 修复
 - 7.1 适用场景
 - 7.2 核心原则
 - 7.3 操作步骤

- 8. SOP D: 测试与 QA
 - 8.1 自动化测试生成: `cx-test`
 - 8.2 测试框架映射
 - 8.3 手工 QA 规划: `cx-qa`
 - 8.4 Verify 所需测试证据
- 9. SOP E: 文档、图表和合规检查
 - 9.1 生成文档: `cx-doc`
 - 9.2 生成图表: `cx-diagram`
 - 9.3 文档模板合规检查: `cx-doc-check`
 - 9.4 文档 SOP 验收标准
- 10. SOP F: 基于参考项目复制新项目
 - 10.1 适用场景
 - 10.2 操作命令
 - 10.3 操作原则
- 11. CodeReview SOP
 - 11.1 审查范围选择
 - 11.2 规则来源优先级
 - 11.3 问题级别
 - 11.4 评分规则
 - 11.5 CodeReview 报告验收清单
- 12. Verify SOP
 - 12.1 Verify 与 CodeReview 的边界
 - 12.2 Verify 前置条件
 - 12.3 五维验证模板
 - 12.4 Verify 失败处理
- 13. 技术栈专项 SOP
 - 13.1 C++ / MES
 - 13.2 VB.NET / EAP
 - 13.3 Java
 - 13.4 Vue / TypeScript
 - 13.5 Python
 - 13.6 C#
- 14. Prompt 使用规范
 - 14.1 启动新需求的标准提示
 - 14.2 批准设计后的标准提示
 - 14.3 CodeReview 标准提示
 - 14.4 Verify 标准提示
 - 14.5 Bug 修复标准提示
 - 14.6 处理 CodeReview 反馈提示
- 15. 常见异常与处理
 - 15.1 Hook 阻止写代码
 - 15.2 `cx-work` 找不到 SPEC-ID
 - 15.3 CodeReview 通过但 Verify 失败

- 15.4 Verify 输出 `insufficient_evidence`
- 15.5 Agent 输出无法写入状态
- 15.6 Windows PowerShell 执行问题
- 16. 安全与合规要求
 - 16.1 代码安全
 - 16.2 数据安全
 - 16.3 Agent 边界
 - 16.4 Git 操作边界
- 17. 交付物验收清单
 - 17.1 需求设计阶段
 - 17.2 编码执行阶段
 - 17.3 CodeReview 阶段
 - 17.4 Verify 阶段
 - 17.5 收尾阶段
- 18. 命令速查表
 - 18.1 标准流程命令
 - 18.2 辅助命令
- 19. 企业落地建议
 - 19.1 试点阶段
 - 19.2 推广阶段
 - 19.3 度量指标
- 20. 最终结论

CX-Aicode 插件 AI 编程标准作业流程 SOP（修订版）

0. 文档定位

本文是一份面向企业内部 AI 编程落地的标准作业流程（SOP），适用于使用 `cx-aicode` 插件在 Claude Code CLI 中进行需求分析、设计、编码、测试、审查、验证、收尾和归档。

本文基于本次上传的 `cx-aicode(5).zip` 代码包进行分析，版本信息如下：

项目	分析结论
插件名称	cx-aicode
版本	2.3.6
更新时间	2026-05-08
项目定位	企业级 Claude Code 插件库，用于生成和治理其他项目，不是业务项目本身
命令数量	26 个 commands/*.md
流程技能数量	26 个 skills/process/*/SKILL.md
插件级只读 Agent	8 个 agents/*.md
Hooks	SessionStart、PreToolUse、UserPromptSubmit、PostToolUse、PostToolUseFailure、PermissionDenied、Stop、StopFailure
支持技术栈	C++、Java、Vue、Python、C#、VB.NET，另含 MES/EAP 业务技能
关键治理机制	设计批准硬门禁、cx-work 三文件机制、Agent-first 只读审查、CodeReview Gate、Verify Gate、状态索引双写
已执行验证	JS 状态/Agent/e2e 测试通过；解析器与 cx-test 测试通过；Bash Hooks 回归测试通过，32 passed / 1 skipped

0.1 本次修订说明

本修订版根据“代码与原 SOP 对比”的结果进行了校正，重点解决三类问题：

- 纠正会导致命令无法按预期执行的写法：**例如 cx-spec 补充 new 子命令，cx-qa 补充 --plan/--cases/--regression 等模式，cx-doc-check 补充 --doc 与 --template，cx-project-copy 补充 --ref 与 --new。
- 区分自动命令能力与流程/提示词能力：**cx-analyze --generate=doc/diagram/test、cx-test、cx-doc、cx-diagram 的产物应作为草案、骨架或辅助材料，不能直接视为已完成的生产级文档、图表或测试。
- 明确审查和验证的边界：**cx-codereview、cx-verify 是基于证据的 Agent-first 审查/验收流程，不替代编译器、CI、SAST/DAST、人工业务验收或正式发布审批。

后续团队执行 SOP 时，应优先使用本文中的“修订后命令”，不要沿用旧版 SOP 中缺少必要参数的简写命令。

本文不是普通“命令说明书”，而是把插件代码中的真实流程、门禁、状态文件、报告脚本和 Hooks 约束整理成可执行 SOP。团队应将本文作为日常 AI 编程操作基线，并可按企业内部项目管理制度进一步细化。

1. 插件架构与 SOP 设计依据

1.1 插件的核心设计思想

cx-aicode 不是单纯的提示词集合，而是一套“流程治理 + 技术栈规范 + 只读 Agent 审查 + Hook 硬门禁 + 状态持久化”的企业级 AI 编程插件。它的核心目标是让 AI 编程从“聊天式写代码”变成“有需求、有设计、有任务、有审查、有验证、有状态、有证据”的工程化流程。

其架构可以概括为四层：

层级	目录/文件	作用
命令入口层	commands/*.md	用户可调用的 slash 命令入口，例如 /cx-work、/cx-codereview、/cx-verify
流程技能层	skills/process/*/SKILL.md	具体流程规则、执行步骤、门禁条件、异常处理
只读 Agent 层	agents/*.md	专门执行分析、审查、验证判断，声明只允许 Read, Grep, Glob
脚本与 Hook 层	hooks/*、scripts/agent/*、scripts/win/*	自动门禁、报告写入、状态更新、初始化、升级、跨会话恢复

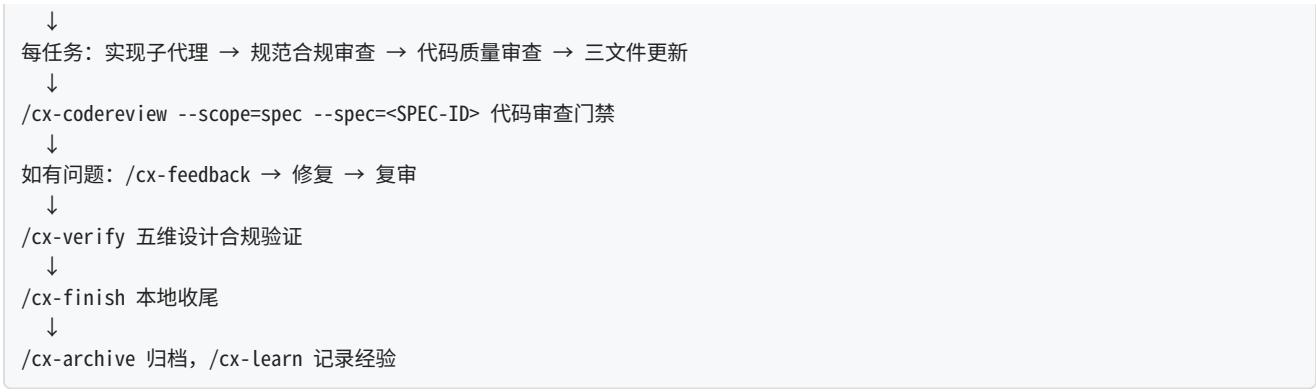
其中最重要的工程化思想是：

- 先设计，再编码：**cx-brainstorm 和 cx-spec 生成 design.md 与 tasks.md，设计未批准前禁止写实现代码。
- 逐任务执行，不一次性大改：**cx-work 以任务为单位派发实现子代理，每个任务结束必须更新三文件。
- 两阶段任务内审查：**先做规范合规审查，再做代码质量审查，顺序不可颠倒。
- workflow 级质量门禁：**所有任务完成后必须运行 /cx-codereview --scope=spec，之后才能运行 /cx-verify。
- Agent-first 但只读：**Agent 只负责判断，不负责写报告、不更新状态、不改代码，避免审查者与执行者职责混淆。
- 状态可恢复：**.cx/status/index.json 和 .cx/work/* 使工作可以跨会话恢复。
- 证据优先：**没有测试、构建、审查、验证报告，不能声称完成。

1.2 标准主流程

标准 AI 编程流程如下：





该流程中最关键的禁止事项：

- 禁止在设计未批准前写实现代码。
- 禁止跳过 cx-work 的任务内两阶段审查。
- 禁止 cx-codereview 未通过就进入 cx-verify。
- 禁止 cx-verify 未通过就进入 cx-finish。
- 禁止只凭 AI 文字声明“已完成”，必须以报告、测试、状态文件作为证据。

1.3 功能边界与可执行性原则

为了避免把“流程能力”误解为“完全自动化能力”，执行本 SOP 时必须遵守以下边界：

功能	可直接依赖的能力	不能直接假设的能力	SOP 执行要求
cx-spec	创建规范目录、导入需求源、生成规范文件结构	自动生成完整可批准设计	必须补充 new 子命令，并由 AI/人工继续填充、评审、批准 design.md 与 tasks.md
cx-analyze --generate=doc/diagram/test	生成结构化草案、模板化文档/图表/测试骨架	真实调用链、完整业务断言、可直接投产文档	产物必须人工复核、补全、运行验证
cx-test	生成测试工件和测试骨架	保证测试代码即刻可运行、业务断言完整	必须执行测试命令，保存日志或 CI 证据
cx-qa	生成测试计划、手工用例、回归套件、Figma 校验或缺陷报告草案	生成可执行自动化测试代码	必须指定 --plan、--cases、--regression、--figma 或 --bug 模式
cx-doc-check	检查指定文档是否符合指定模板	自动知道要检查哪个文档和模板	必须传入 --doc=<文档> 与 --template=<模板>
cx-project-copy	基于参考项目复制并做差异化替换	在无路径、无映射确认时自动复制	必须指定 --ref 与 --new，替换映射必须人工确认
cx-codereview	基于规则和证据进行只读代码审查	替代编译器、SAST、人工架构评审	L1/L2 阻断项必须修复或形成明确豁免记录
cx-verify	验证需求覆盖、接口、数据、业务规则、异常处理	替代测试执行和业务验收	必须提供新鲜测试/构建/审查证据

一句话原则：插件可以强制流程、生成草案、组织证据和执行审查，但不能替代真实编译、真实测试、人工审批和业务验收。

1.4 插件目录与产物目录

插件安装目录与业务项目目录需要区分。

插件自身目录包含：

commands/	# slash 命令入口
skills/process/	# 流程技能
skills/tech/	# 技术栈规范
skills/quality/	# 技术栈质量规则
skills/test/	# 技术栈测试规则
skills/business/	# MES/EAP 业务技能
skills/core/code-analysis/	# 代码分析引擎
agents/	# 插件级只读 Agent
hooks/	# 跨平台 Hooks
scripts/agent/	# Agent 输出校验、报告、状态脚本
scripts/win/	# 初始化、升级、OpenSpec 等 PowerShell 脚本
reference/	# CodeReview 规则与评分配置
templates/	# 文档、图表模板
flavors/	# MES/EAP 初始化模板

业务项目初始化后会生成：

```
.cx/
├── config.json
├── status/
│   └── index.json
├── specs/
│   └── <SPEC-ID>/
│       ├── source.md
│       ├── sources/
│       ├── metadata.yaml
│       ├── design.md
│       └── tasks.md
├── work/
│   ├── task_plan.md
│   ├── findings.md
│   ├── progress.md
│   └── cx-work-session.json
├── reports/
│   ├── codereview/
│   ├── verify/
│   ├── qa/
│   ├── doc-check/
│   └── analyze/
└── quality/
```

SOP 的重点就是围绕这些文件和目录进行可追溯操作。

2. 角色与职责

2.1 角色定义

角色	职责
需求负责人	提供需求文档、确认需求边界、批准设计方案
AI 编程操作员	调用插件命令，驱动 AI 进行设计、实现、测试和报告生成
技术负责人	审查架构、关键设计、技术债、风险问题，决定 conditional approval 是否成立
代码审查负责人	处理 CodeReview 报告中的 L1/L2/L3/L4 问题，确认修复或豁免
QA/测试负责人	基于 cx-test/cx-qa 产物制定测试计划，确认测试证据新鲜有效
发布/收尾负责人	确认 Review Gate、Verify Gate、测试均通过后执行收尾和归档

2.2 职责分离原则

插件代码已经明确区分“实现者”和“审查者”：

- 实现子代理可以写代码、跑测试、修复问题。
- `cx-work-spec-reviewer`、`cx-work-quality-reviewer`、`cx-code-reviewer`、`cx-verifier` 等 Agent 只能读文件和给出判断。
- 报告写入与状态更新由主流程和 `scripts/agent/*` 完成。

因此团队在使用时也应遵守职责分离：

1. 不要让同一个 AI 子任务既写代码又批准自己。
2. 不要让审查 Agent 修改代码。
3. 不要人工口头覆盖 L1/L2 阻断项，除非形成明确审批记录。
4. 不要用“看起来没问题”替代测试和构建证据。

3. 环境准备与安装 SOP

3.1 前置环境

使用前必须确认以下环境：

类别	要求
Claude Code CLI	已安装并可正常运行
Node.js	必需，插件中的测试生成器、Agent 报告脚本、Hook dispatcher 均依赖 Node.js
Git	必需，用于变更范围识别、worktree、分支检查
Windows PowerShell	Windows 环境建议 PowerShell 5.1，脚本中明确要求 #requires -Version 5.1
Git Bash / Bash	Windows 下建议配合 Git Bash 使用，Bash Hooks 也有完整实现
语言构建工具	Java/Maven 或 Gradle、Node/Vite、Python/pytest、.NET、CMake/gtest 等按项目技术栈准备
编码	PowerShell/Batch 使用 UTF-8 with BOM + CRLF；Markdown/JSON/JS 默认 UTF-8 LF

3.2 插件安装

在获取插件源码后进入插件根目录：

```
cd /path/to/cx-aicode
claude plugin marketplace add ./
claude plugin install cx-aicode
claude agents
```

检查点：

- `claude agents` 中应能看到 8 个 `cx-*` Agent。
- 插件版本应为 2.3.6。
- 不要在插件根目录误执行 `/cx-init`。插件自身不是被治理业务项目。

3.3 业务项目初始化

在业务项目根目录执行初始化。

MES 项目：

```
/cx-aicode:cx-init --flavor=mes --name=<项目名>
```

EAP 项目：

```
/cx-aicode:cx-init --flavor=eap --name=<项目名>
```

通用技术栈项目：

```
/cx-aicode:cx-init --tech=java,vue --name=<项目名>
/cx-aicode:cx-init --tech=python --name=<项目名>
/cx-aicode:cx-init --tech=csharp --name=<项目名>
```

最小初始化：

```
/cx-aicode:cx-init --minimal --name=<项目名>
```

初始化后必须检查：

```
/cx-aicode:cx-status  
/cx-aicode:cx-doctor
```

检查点：

- `.cx/config.json` 存在。
- `.cx/status/index.json` 存在。
- 项目 `CLAUDE.md` 已写入 CX-Aicode 区块。
- Hooks 能自动生效。
- `reviewStatus`、`verifyStatus` 初始状态合理，例如 `not_required` 或空状态。

3.4 初始化方式选择

场景	推荐初始化
半导体 MES C++ 项目	<code>--flavor=mes</code>
EAP / SECS / HSMS VB.NET 项目	<code>--flavor=eap</code>
Java 后端项目	<code>--tech=java</code>
Vue 前端项目	<code>--tech=vue</code>
Java + Vue 前后端项目	<code>--tech=java,vue</code>
Python 项目	<code>--tech=python</code>
C# 项目	<code>--tech=csharp</code>
不确定或先试点	<code>--minimal</code> ，后续补充组件

4. 状态文件与门禁规则

4.1 `.cx/status/index.json` 的作用

`.cx/status/index.json` 是跨会话状态持久化核心。它记录：

- 当前 active spec / change。
- CodeReview 状态。
- Verify 状态。
- 报告路径。
- 质量门禁详情。
- 上次会话任务进度。

- 中断恢复信息。

典型结构：

```
{
  "reviewStatus": "passed|conditional|conditional_approved|failed",
  "verifyStatus": "passed|conditional|failed|insufficient_evidence",
  "codeReview": {
    "scope": "spec",
    "specId": "MES-0535",
    "reportPath": ".cx/reports/codereview/CodeReview-YYYYMMDD-HHmss.md",
    "score": 86,
    "grade": "B",
    "issueCounts": { "L1": 0, "L2": 0, "L3": 4, "L4": 7 }
  },
  "verification": {
    "specId": "MES-0535",
    "reportPath": ".cx/reports/verify/verification-report-YYYYMMDD-HHmss.md"
  },
  "qualityGates": {
    "codeReview": { "status": "passed" },
    "verify": { "status": "passed" }
  }
}
```

cx-finish 读取顶层 reviewStatus 和 verifyStatus，Hooks 会同时读取 qualityGates。因此状态更新必须保持双写一致。

4.2 CodeReview 状态规则

插件代码中的 scripts/agent/update-status.js 对 CodeReview 状态有硬校验。

条件	状态	是否可进入 Verify
L1 > 0	failed	否
L2 > 0	默认 failed	否，除非负责人明确接受并形成 conditional_approved
L1=0 且 L2=0，分数达标	passed	是
存在 L3/L4 且需后续跟踪	conditional	否，需先通过 cx-feedback 关闭、修复或审批
L2 已被负责人显式接受	conditional_approved	是，但必须记录审批理由或审批文件

conditional_approved 必须满足：

- 不允许有 L1 问题。
- 必须提供 --approval-reason 或 --approval-file。
- 如果提供审批文件，文件必须存在且位于项目目录内。

4.3 Agent 输出契约

v2.3.6 的只读 Agent 需要输出 cx-agent-result 代码块，随后由 validate-agent-result.js 解析和校验。

必须包含：

```
{
  "agent": "cx-code-reviewer",
  "status": "passed|conditional|failed|insufficient_evidence",
  "summary": "...",
  "riskLevel": "L0|L1|L2|unknown",
  "issueCounts": { "p0": 0, "p1": 0, "p2": 0 },
  "issues": [],
  "nextAction": "allow_next_step|require_fix|require_human_review|insufficient_evidence"
}
```

兼容旧字段时，p0/p1/p2 会映射到旧版 L1/L2/L3，并可额外统计 L4。SOP 上应统一以 L1-L4 进行管理，因为 .cx/status/index.json 中的 CodeReview 结果最终写入 issueCounts.L1/L2/L3/L4。

4.4 .cx/work/ 三文件机制

cx-work 使用三文件追踪任务进度：

文件	作用
.cx/work/task_plan.md	任务计划、Phase、checkbox 状态
.cx/work/findings.md	规范摘要、发现、决策、CodeReview 问题摘录
.cx/work/progress.md	会话日志、完成记录、变更文件列表、质量门禁记录
.cx/work/cx-work-session.json	有效会话 marker，包含 createdBy、specId、initializedAt

注意：仅手工创建三文件不能解锁编码门禁。cx-work-session.json 必须有效，且三文件中的 SPEC-ID 必须与 session marker 一致。

每个任务完成后必须执行：

1. 更新 task_plan.md checkbox：- [] → - [x]。
2. 更新 progress.md Phase Log，补充变更文件和任务完成记录。
3. 重新读取 task_plan.md，确认下一任务在上下文中。

这是跨会话恢复和后续质量门禁读取进度的基础。

4.5 Hook 门禁机制

插件注册的 Hooks 不是装饰性能力，而是流程安全措施。

Hook	SOP 中的意义
SessionStart	会话开始时恢复 .cx/status/index.json 和有效 cx-work 上下文
PreToolUse	在写文件、执行命令前进行准入检查；设计未批准或无有效 cx-work marker 时阻止实现写入
UserPromptSubmit	有效 cx-work 会话下注入当前任务上下文
PostToolUse	记录工具使用、文件变更、任务完成线索
PostToolUseFailure / PermissionDenied	记录失败和错误上下文
Stop	会话结束时更新中断恢复状态
StopFailure	记录异常结束

SOP 要点：

- 不要试图绕开 Hook 写代码。
- 如果 Hook 阻止写入，优先检查 design 是否已批准、cx-work-session.json 是否存在、SPEC-ID 是否一致。
- Hook 可以辅助记录，但不能替代主流程主动更新三文件。

5. SOP A：新需求 / 新功能开发

5.1 适用场景

适用于以下情况：

- 新增业务功能。
- 新增模块、接口、页面、报表、设备接入能力。
- 需求尚需澄清，不能直接编码。
- 需要形成完整设计和任务清单。

5.2 输入物

输入	要求
需求 ID	推荐使用外部需求编号，例如 MES-0535、OPT-0123
需求类型	产品需求 或 优化需求
需求说明文件	Markdown、Word 转 Markdown、PDF 摘要、需求截图说明等，建议先转为可读文本
项目上下文	当前代码库、架构说明、接口文档、数据表、业务约束

5.3 操作步骤

Step 0: 确认项目已初始化

```
/cx-aicode:cx-status  
/cx-aicode:cx-doctor
```

若项目未初始化：

```
/cx-aicode:cx-init --flavor=mes --name=<项目名>
```

或：

```
/cx-aicode:cx-init --tech=java,vue --name=<项目名>
```

验收标准：

- `.cx/` 已存在。
- 项目 `CLAUDE.md` 包含 CX-Aicode 区块。
- 技术栈与业务 flavor 正确。

Step 1: 登记需求并启动 Brainstorm

推荐命令：

```
/cx-aicode:cx-brainstorm --spec-id MES-0535 --source docs/需求说明.md --type 产品需求
```

多文件需求：

```
/cx-aicode:cx-brainstorm --spec-id MES-0535 --source docs/a.md,docs/b.md --type 优化需求
```

执行后应生成：

```
.cx/specs/MES-0535/  
├── source.md  
├── sources/  
└── metadata.yaml
```

验收标准：

- SPEC-ID 合法，只包含字母、数字、点、下划线、短横线，并以字母或数字开头。
- 原始需求文件已复制到 `sources/`。
- `source.md` 成为固定入口。
- `metadata.yaml` 记录需求类型和来源。

Step 2: 澄清需求并形成设计

AI 操作员应要求 AI：

请基于 `.cx/specs/MES-0535/source.md` 进行需求澄清，先列出不明确点，再给出可执行设计方案。
不得编写实现代码。

必须输出：

`.cx/specs/MES-0535/design.md`
`.cx/specs/MES-0535/tasks.md`

`design.md` 至少应包含：

- 背景和目标。
- 功能范围。
- 非功能约束。
- 接口契约。
- 数据模型。
- 业务规则。
- 异常处理。
- 安全与权限。
- 测试策略。
- 风险与未决问题。

`tasks.md` 至少应包含：

- 可执行任务列表。
- 任务依赖关系。
- 每个任务的验收条件。
- 测试任务。
- 文档/配置任务。

Step 3: 设计审批硬门禁

需求负责人和技术负责人必须审阅 `design.md` 与 `tasks.md`。审批通过后，在 `design.md` 中写入明确批准标记，例如：

```
## Approval

**APPROVED** by <负责人> at 2026-05-12.
```

验收标准：

- `design.md` 中存在 `## Approval`。
- 存在 `**APPROVED**` 明确标记。
- 设计中的未决问题已经关闭或转为任务。

未批准前禁止执行：

`/cx-aicode:cx-work`

Step 4: 进入 cx-work 执行

指定 SPEC-ID 执行：

```
/cx-aicode:cx-work --spec=MES-0535
```

插件应初始化：

```
.cx/work/task_plan.md  
.cx/work/findings.md  
.cx/work/progress.md  
.cx/work/cx-work-session.json
```

验收标准：

- task_plan.md 文件头包含 SPEC-ID: MES-0535。
- cx-work-session.json 中 createdBy 为 cx-work。
- cx-work-session.json 中 specId 为 MES-0535。
- 三文件与当前 SPEC-ID 一致。

Step 5: 逐任务执行循环

每个任务按以下顺序执行：

```
实现子代理  
↓  
运行或生成测试，完成自检  
↓  
规范合规审查 Agent  
↓  
如不通过，修复并重新审查  
↓  
代码质量审查 Agent  
↓  
如不通过，修复并重新审查  
↓  
更新三文件  
↓  
读取 task_plan.md 进入下一任务
```

实现子代理状态处理规则：

子代理状态	处理
DONE	进入规范合规审查
DONE_WITH_CONCERNS	先分析 concerns；若影响正确性或范围，修复后再审查
NEEDS_CONTEXT	补充上下文后重新派发
BLOCKED	判断是上下文不足、模型不足、任务过大还是计划错误；不得盲目重试

两阶段审查顺序：

1. cx-work-spec-reviewer：检查“是否构建了正确的东西”。

2. cx-work-quality-reviewer：检查“是否良好构建”。

禁止事项：

- 禁止并行执行多个实现子代理修改共享代码。
- 禁止规范合规审查未通过就做质量审查。
- 禁止审查发现问题后直接进入下一任务。
- 禁止用 `TodoWrite` 替代 `task_plan.md` checkbox。

Step 6: 每任务完成后的强制记录

每个任务通过两阶段审查后，主流程必须立即执行：

1. 修改 `task_plan.md`：当前任务 - [] 改为 - [x]。
2. 修改 `progress.md`：记录任务完成、测试结果、变更文件。
3. 读取 `task_plan.md`：确认下一任务。

建议在 `progress.md` 中记录：

```
## Phase Log

### Task 1.1 完成
- 时间: 2026-05-12 10:30
- 变更文件:
  - src/services/EquipmentStateService.java
  - src/services/EquipmentStateServiceTest.java
- 测试: mvn test 通过, 新增 8 个测试
- 规范审查: 通过
- 质量审查: 通过
```

Step 7: 全部任务完成后运行 Spec 范围 CodeReview

```
/cx-aicode:cx-codereview --scope=spec --spec=MES-0535 --tech=auto
```

日常需求开发不建议默认全量审查。全量审查用于发布前、质量基线或专项治理。Spec 范围审查可以避免历史问题混入本次任务结论。

CodeReview 应生成：

```
.cx/reports/codereview/CodeReview-YYYYMMDD-HHmss.md
```

并更新：

```
.cx/status/index.json
```

验收标准：

- `reviewStatus` 为 `passed` 或 `conditional_approved`。
- 报告路径存在。
- `issueCounts` 包含 L1/L2/L3/L4。
- L1 必须为 0。
- L2 如未修复，必须有明确 `conditional approval`。

Step 8: 处理 CodeReview 问题

如果 reviewStatus=failed 或 conditional:

```
/cx-aicode:cx-feedback --from-codereview --to-task-plan
```

处理规则:

- L1: 必须修复, 不允许豁免。
- L2: 默认必须修复; 如确需接受, 必须形成审批理由或审批文件。
- L3: 应修复或转入后续技术债清单。
- L4: 可视项目规范决定是否修复。

当问题较多时, 应追加到 task_plan.md 的 CodeReview Fix Phase, 不要把完整报告复制进计划文件。

修复后重新运行同一范围审查:

```
/cx-aicode:cx-codereview --scope=spec --spec=MES-0535 --tech=auto
```

Step 9: 运行 Verify Gate

CodeReview 通过后运行:

```
/cx-aicode:cx-verify
```

Verify 关注的是“实现是否满足 spec/design/tasks”, 不是重复做代码质量审查。

五维验证应覆盖:

维度	检查内容
需求覆盖	tasks.md 与实现是否一一对应
接口契约	API、参数、返回值、错误码是否符合设计
数据模型	表、字段、实体、DTO、迁移是否一致
业务规则	状态机、权限、流程、边界条件是否正确
异常处理	错误路径、超时、空值、回滚、日志是否覆盖

必须提供新鲜测试或构建证据。cx-verifier Agent 本身不运行测试, 主流程必须先收集证据。没有证据时应输出 insufficient_evidence, 不得写入 verifyStatus=passed。

验收标准:

- .cx/reports/verify/verification-report-*.md 存在。
- .cx/status/index.json.verifyStatus=passed。
- 如有 conditional 或 failed, 必须修复并复验。

Step 10: 本地收尾

```
/cx-aicode:cx-finish
```

cx-finish 会检查：

- reviewStatus 是否为 passed 或 conditional_approved。
- verifyStatus 是否为 passed。
- 测试是否通过。

收尾只提供两个本地选项：

1. 保留分支。
2. 丢弃工作。

插件明确不应主动执行 git merge、git commit、git push。这些操作应由企业内部版本管理流程单独处理。

Step 11: 归档与经验沉淀

需求完成后执行：

```
/cx-aicode:cx-archive
```

遇到有价值的问题、模式或反模式时执行：

```
/cx-aicode:cx-learn --type error "问题描述 / 解决方案"
```

归档内容应包含：

- 需求源文件。
- 设计与任务。
- CodeReview 报告。
- Verify 报告。
- QA/测试报告。
- 关键经验与后续技术债。

6. SOP B: 已有项目增量变更 / Brownfield 修改

6.1 适用场景

- 在既有系统中新增字段、接口、配置项。
- 修改已有 API 或页面逻辑。
- 调整业务流程或设备接入规则。
- 需求相对明确，不需要长时间 Brainstorm。

6.2 推荐流程

```
/cx-aicode:cx-analyze --tech=auto 了解现有结构
↓
/cx-aicode:cx-spec new --delta --spec-id <SPEC-ID> --source <file> 生成 Delta Spec
↓
审查 delta-spec.md / design.md / tasks.md
↓
批准后 /cx-work --spec=<SPEC-ID>
↓
/cx-codereview --scope=spec
↓
/cx-verify
↓
/cx-finish
```

6.3 操作步骤

Step 1: 先做代码结构盘点

```
/cx-aicode:cx-analyze --tech=auto --output=markdown
```

如果需要输出结构辅助：

```
/cx-aicode:cx-analyze --generate=diagram
/cx-aicode:cx-analyze --generate=doc
```

重要边界：

- Python 解析器使用 AST，精度较高。
- Java/Vue/C++/VB.NET/C# 主要是正则或轻量扫描，适合结构概览，不应作为严格审计结论。
- 生成的文档、图表、测试多为模板驱动草案，必须人工复核；不得把这些产物直接作为真实调用链、完整 API 文档或可执行测试结论。

Step 2: 创建 Delta Spec

简单增量变更建议使用 /cx-spec 的 Delta Spec 模式。

```
/cx-aicode:cx-spec new --spec-id OPT-0123 --delta --source docs/优化需求.md
```

Delta Spec 推荐包含：

```
## ADDED Requirements
### Requirement: 新增需求名
Scenario: 场景名
GIVEN: 前置条件
WHEN: 触发动作
THEN: 预期结果

## MODIFIED Requirements
### Requirement: 修改需求名 [MODIFIED]
```

变更描述

REMOVED Requirements

Requirement: 删除需求名 [REMOVED]

删除原因

RENAMED Artifacts

Artifact: 原名称 → 新名称

重命名原因

Step 3: 转换为可执行任务

如果存在 `delta-spec.md` 但没有 `tasks.md`，`cx-work` 会按规则从 Delta Spec 转换任务。建议仍人工检查：

- 新增需求是否有实现任务。
- 修改需求是否有回归测试任务。
- 删除项是否有兼容性和清理任务。
- 重命名是否有迁移任务。

Step 4: 按标准 `cx-work` 执行

```
/cx-aicode:cx-work --spec=OPT-0123
```

每个任务仍必须经过：

实现 → 规范合规审查 → 代码质量审查 → 三文件更新

Step 5: 按 Spec 范围审查

```
/cx-aicode:cx-codereview --scope=spec --spec=OPT-0123 --tech=auto
```

也可以使用分支变更模式：

```
/cx-aicode:cx-codereview --scope=changed --base=origin/main --tech=auto
```

选择原则：

情况	推荐 scope
有明确 SPEC-ID	spec
没有规范，但已在分支上修改	changed
发布前质量基线	full
指定时间或 commit 范围	since / commit 增量模式

7. SOP C: Bug 修复

7.1 适用场景

- 测试失败。
- 线上缺陷。
- 用户反馈功能异常。
- 代码行为不符合预期。

7.2 核心原则

Bug 修复不能直接“猜一个修复方案”。必须先找到根因。cx-debug 的核心是四阶段：

1. 根本原因调查。
2. 模式分析。
3. 假设与测试。
4. 实现修复。

7.3 操作步骤

Step 1: 启动调试流程

```
/cx-aicode:cx-debug
```

给 AI 的任务描述应包含：

```
问题现象：...
复现步骤：...
期望结果：...
实际结果：...
相关日志：...
最近变更：...
请使用 cx-debug 四阶段流程，先定位根因，不要直接修改代码。
```

Step 2: 根因调查

AI 必须先收集：

- 失败测试或日志。
- 相关代码路径。
- 最近变更。
- 输入输出差异。
- 配置和环境差异。

禁止事项：

- 没有复现就修复。
- 没有证据就猜测根因。
- 只改表面报错，不分析调用链。

Step 3: 写失败测试

除非用户明确 `--no-tdd`，否则 bug 修复应遵循 TDD：

```
RED: 写一个能复现 bug 的失败测试
GREEN: 写最小修复代码
REFACTOR: 清理代码，保持测试通过
```

常见命令：

```
/cx-aicode:cx-tdd
```

Step 4: 实施修复

修复应最小化：

- 不引入无关功能。
- 不重构与 bug 无关的大范围代码。
- 不隐藏异常。
- 不删除失败测试。

Step 5: 审查和验证

如果 bug 修复有 SPEC-ID：

```
/cx-aicode:cx-codereview --scope=spec --spec=<SPEC-ID> --tech=auto
/cx-aicode:cx-verify
```

如果是临时分支修复：

```
/cx-aicode:cx-codereview --scope=changed --base=origin/main --tech=auto
```

验收标准：

- 失败测试先失败，修复后通过。
- 原有测试回归通过。
- 根因记录在 `findings.md` 或缺陷报告中。
- CodeReview 不存在 L1/L2 阻断项。
- Verify 或等效验证证据通过。

8. SOP D：测试与 QA

8.1 自动化测试生成：cx-test

cx-test 可生成单元、集成、E2E 测试工件。

常用命令：

```
/cx-aicode:cx-test --type=unit --lang=java
/cx-aicode:cx-test --type=integration --lang=python
/cx-aicode:cx-test --type=e2e --lang=vue
/cx-aicode:cx-test --type=all --lang=java
```

自动检测语言时：

```
/cx-aicode:cx-test --type=all
```

注意：如果无法识别技术栈，必须显式指定 `--lang=java|vue|python|csharp|cpp|vbnet`。

重要边界：cx-test 的部分生成内容可能是测试骨架、stub 或带 TODO 的方法体。执行 SOP 时必须补全测试数据、Mock、断言和环境配置，并实际运行测试。只有“测试执行结果”才能作为 Verify 证据，单纯生成测试文件不能作为通过依据。

8.2 测试框架映射

技术栈	单元测试	集成测试 / E2E
Java	JUnit 5 + Mockito	Spring Boot Test
Vue	Vitest	Vitest + MSW / Playwright
Python	pytest	pytest + requests / ASGI transport
C#	xUnit	WebApplicationFactory
VB.NET	NUnit	DB / 设备模拟测试
C++	Google Test	仿真器 / 集成测试

8.3 手工 QA 规划：cx-qa

当需要测试计划、手工用例、回归套件、Figma 设计校验或缺陷报告时使用。--write 只表示写入报告，不代表 QA 类型；必须同时指定具体模式。

常用命令：

```
/cx-aicode:cx-qa --plan --feature=<功能名> --write
/cx-aicode:cx-qa --cases --feature=<功能名> --count=<N> --write
/cx-aicode:cx-qa --regression --feature=<功能名> --write
```



```
/cx-aicode:cx-qa --figma --url=<FigmaURL> --write  
/cx-aicode:cx-qa --bug --desc=<缺陷描述> --write
```

cx-qa 不生成可执行测试代码，不影响 reviewStatus / verifyStatus，产物应作为测试计划或手工测试草案。

应输出：

```
.cx/reports/qa/qa-plan-YYYYMMDD-HHmss.md  
.cx/reports/qa/qa-cases-YYYYMMDD-HHmss.md  
.cx/reports/qa/regression-suite-YYYYMMDD-HHmss.md  
.cx/reports/qa/bug-report-YYYYMMDD-HHmss.md
```

QA 计划至少包含：

- 测试范围。
- 用例清单。
- 前置数据。
- 操作步骤。
- 预期结果。
- 回归范围。
- 风险区域。

8.4 Verify 所需测试证据

cx-verify 不能只看 AI 自述。必须提供新鲜证据：

- 构建日志。
- 单元测试结果。
- 集成测试结果。
- E2E 测试结果。
- 手工 QA 执行记录。
- 关键业务场景验证截图或日志。

证据判定标准：

证据	是否可用
当前会话刚执行的测试输出	可用
当天同一 commit 的 CI 报告	可用
旧分支旧版本测试报告	不可直接使用
AI 声称“应该通过”	不可用
仅生成测试但未执行	不足以通过 Verify

9. SOP E：文档、图表和合规检查

9.1 生成文档：cx-doc

适用于生成或更新：

- 需求文档。
- 设计文档。
- 用户手册。
- API 文档。
- 架构说明。
- 运维文档。

命令：

```
/cx-aicode:cx-doc
```

可结合代码分析：

```
/cx-aicode:cx-analyze --generate=doc
```

注意：模板生成内容必须人工校对，不能直接视为真实代码文档。对于 `cx-analyze --generate=doc` 生成的 API 示例、模块说明和流程描述，应由开发负责人对照真实代码逐项确认。

9.2 生成图表：cx-diagram

适用于生成：

- 架构图。
- 流程图。
- 时序图。
- 类图。
- 状态图。
- MES/EAP 特定图表。

命令：

```
/cx-aicode:cx-diagram  
/cx-aicode:cx-analyze --generate=diagram
```

注意：`cx-analyze --generate=diagram` 生成的图表可能不包含真实方法调用链或完整运行时依赖关系，应作为架构沟通草图使用，正式设计图必须人工复核。

图表模板位于：

```
templates/diagrams/  
templates/diagrams/mes/  
templates/diagrams/eap/
```

9.3 文档模板合规检查：cx-doc-check

当需要检查文档是否符合模板时，必须明确指定待检查文档和模板。模板可以是内置模板名，也可以是自定义模板路径。

```
/cx-aicode:cx-doc-check --doc=<待检查文档> --template=<模板名或模板路径>
```

示例：

```
/cx-aicode:cx-doc-check --doc=D:\docs\需求文档.md --template=requirements  
/cx-aicode:cx-doc-check --doc=D:\docs\用户手册.md --template=user-manual  
/cx-aicode:cx-doc-check --doc=D:\docs\蚀刻机规格书.md --template=D:\templates\设备规格书.md
```

应输出：

```
.cx/reports/doc-check/doc-check-report-YYYYMMDD-HHmss.md
```

cx-doc-check 使用 cx-doc-compliance-reviewer 只读 Agent，Agent 只判断文档结构和模板字段，不修改文件。报告写入和状态记录由主流程完成。

9.4 文档 SOP 验收标准

文档类型	必须检查
需求文档	需求编号、范围、业务规则、验收标准
设计文档	接口、数据模型、异常处理、测试策略、Approval
用户手册	操作步骤、截图/示意、异常处理、FAQ
API 文档	URL、方法、参数、返回、错误码、鉴权
架构图	模块边界、依赖方向、数据流、部署关系
CodeReview 报告	问题级别、文件行号、影响、建议、评分

10. SOP F：基于参考项目复制新项目

10.1 适用场景

- 半导体设备接入项目相似度高。
- EAP 项目需要从同类设备项目复制并差异化。

- 参考项目结构清晰，可通过替换设备名、模块名、协议参数生成新项目。

10.2 操作命令

```
/cx-aicode:cx-project-copy --ref=<参考项目路径> --new=<新项目路径>
```

示例：

```
/cx-aicode:cx-project-copy --ref=D:\projects\EAP_OldDevice --new=D:\projects\EAP_NewDevice  
/cx-aicode:cx-project-copy --ref=D:\projects\EAP_OldDevice --new=D:\projects\EAP_NewDevice --eqp-id=ETCH01 --  
mdl=ETCH-3000 --ip=192.168.1.50
```

补充输入建议：

新设备名：NewDevice
差异说明：新增 S6F11 事件，修改报警映射，保留 HSMS 连接逻辑
替换映射：由插件扫描候选标识符后，必须人工确认再执行

10.3 操作原则

1. 先扫描参考项目，发现候选标识符。
2. 用户确认替换映射后再执行复制。
3. 复制后必须运行 `cx-analyze`。
4. 必须运行测试生成或测试检查。
5. 必须运行 `cx-codereview` 和 `cx-verify`。

禁止：

- 未确认标识符就批量替换。
- 替换字符串覆盖协议常量、数据库字段或第三方接口字段。
- 复制后不做 CodeReview。

11. CodeReview SOP

11.1 审查范围选择

范围	命令	适用场景
Spec 范围	<code>/cx-codereview --scope=spec --spec=<SPEC-ID></code>	标准需求开发完成后
分支变更	<code>/cx-codereview --scope=changed --base=origin/main</code>	无 SPEC-ID 的变更分支
全量	<code>/cx-codereview --scope=full</code>	发布前、质量基线、专项治理
时间/commit 增量	<code>/cx-codereview --since=...</code>	指定时间或 commit 后的变更

11.2 规则来源优先级

CodeReview 规则应优先使用项目自定义规则，其次使用插件内置规则。插件内置规则位于：

```
reference/<tech>/code_rules.md
reference/<tech>/scoring_rules.yml
```

支持技术栈：

```
cpp, java, vue, python, csharp, vbnet, mes, eap
```

11.3 问题级别

级别	含义	处理要求
L1	阻塞 / 紧急	必须修复，不可豁免
L2	严重 / 高	默认必须修复；如接受风险，需负责人审批
L3	一般 / 中	建议修复，或纳入任务计划
L4	轻微 / 低	可按项目标准处理

11.4 评分规则

评分按维度扣分后加权计算，不是简单从 100 分扣所有问题分。基本逻辑：

1. 每个维度基础分 100。
2. 按问题级别扣除维度分。
3. 单个维度最低为 0。
4. 按维度权重计算总分。
5. 映射等级 A/B/C/D/F。

SOP 要求：

- L1/L2 比总分更重要。
- 即使总分较高，存在 L1 仍然失败。
- conditional_approved 必须有审批证据。

11.5 CodeReview 报告验收清单

报告必须包含：

- 项目路径。
- 技术栈。
- 规则来源。
- 审查范围。

- 审查时间。
- 扫描文件数。
- 发现问题数和 L1-L4 汇总。
- 总分和等级。
- 每个问题的文件路径、行号、代码片段、影响、建议、规则引用。
- 评分明细。
- 状态写入结果。

12. Verify SOP

12.1 Verify 与 CodeReview 的边界

命令	回答的问题	示例
cx-codereview	代码是否安全、可维护、符合技术栈规范	SQL 拼接、空指针、复杂度、命名、硬编码密钥
cx-verify	实现是否真正满足需求和设计	漏实现需求、接口字段不一致、业务规则错误、异常路径缺失

两者不能互相替代。

12.2 Verify 前置条件

运行 Verify 前必须：

- 已完成 cx-work。
- 已运行 Spec 范围 CodeReview。
- reviewStatus=passed 或 conditional_approved。
- 已获得新鲜测试或构建证据。

若 reviewStatus=conditional 或 failed，必须先处理 CodeReview。

12.3 五维验证模板

```
# Verify 报告

## 1. 需求覆盖
- 需求项：...
- 实现证据：...
- 结论：通过/失败/证据不足

## 2. 接口契约
- 设计接口：...
- 实现接口：...
- 测试证据：...
- 结论：...
```

```

## 3. 数据模型
...

## 4. 业务规则
...

## 5. 异常处理
...

## 总结
- status: passed / conditional / failed / insufficient_evidence
- nextAction: allow_next_step / require_fix / require_human_review / insufficient_evidence

```

12.4 Verify 失败处理

Verify 结果	处理
passed	进入 cx-finish
conditional	补充验证或处理条件后复验
failed	回到 cx-work 或 cx-feedback 修复问题
insufficient_evidence	先补充测试/构建/人工验证证据，再运行 Verify

13. 技术栈专项 SOP

13.1 C++ / MES

适用于 SiView/CORBA、SECS/GEM、VS2010+ 等场景。

重点检查：

- C++11 兼容性。
- 内存安全。
- 线程安全。
- CORBA 接口契约。
- SECS/GEM 状态机。
- 异常和日志。
- 老旧编译器兼容。

推荐初始化：

```
/cx-aicode:cx-init --flavor=mcs
```

推荐审查：

```
/cx-aicode:cx-codereview --scope=spec --spec=<SPEC-ID> --tech=mcs
```

13.2 VB.NET / EAP

适用于 HSMS、SECS-I、设备自动化项目。

重点检查：

- .NET Framework 4.5.1+ 兼容性。
- Option Explicit。
- 错误处理。
- 线程与连接状态。
- HSMS 消息处理。
- 设备模拟测试。

推荐初始化：

```
/cx-aicode:cx-init --flavor=eap
```

推荐审查：

```
/cx-aicode:cx-codereview --scope=spec --spec=<SPEC-ID> --tech=eap
```

13.3 Java

重点检查：

- Spring Boot 版本约束。
- MyBatis/MyBatis-Plus 查询安全。
- 事务边界。
- DTO/Entity 分层。
- 异常与日志。
- 单元和集成测试。

推荐：

```
/cx-aicode:cx-test --type=all --lang=java  
/cx-aicode:cx-codereview --scope=spec --spec=<SPEC-ID> --tech=java
```

13.4 Vue / TypeScript

重点检查：

- Vue 3 Composition API。
- TypeScript 类型安全。
- 组件边界。
- Pinia 状态管理。
- API 调用封装。

- Vitest / Playwright 测试。

推荐：

```
/cx-aicode:cx-test --type=e2e --lang=vue  
/cx-aicode:cx-codereview --scope=spec --spec=<SPEC-ID> --tech=vue
```

13.5 Python

重点检查：

- Python AST 分析结果。
- FastAPI 路由。
- Pandas 数据处理。
- 类型提示。
- pytest。
- 异常与日志。

推荐：

```
/cx-aicode:cx-analyze --tech=python  
/cx-aicode:cx-test --type=all --lang=python
```

13.6 C#

重点检查：

- .NET 6/8 或项目指定版本。
- 依赖注入。
- Entity Framework 查询。
- 异步方法。
- xUnit。
- WebApplicationFactory 集成测试。

推荐：

```
/cx-aicode:cx-test --type=all --lang=csharp
```

14. Prompt 使用规范

14.1 启动新需求的标准提示

```
/cx-aicode:cx-brainstorm --spec-id MES-0535 --source docs/需求说明.md --type 产品需求
```

请先读取 `.cx/specs/MES-0535/source.md`，按照 `cx-brainstorm` 流程澄清需求。
在 `design.md` 获得 APPROVED 之前，不要编写任何实现代码。
最终输出 `.cx/specs/MES-0535/design.md` 和 `.cx/specs/MES-0535/tasks.md`。

14.2 批准设计后的标准提示

我已审批 `.cx/specs/MES-0535/design.md`，文件中已写入 `## Approval` 和 `**APPROVED**`。
请使用 `/cx-aicode:cx-work --spec=MES-0535` 开始执行。
必须逐任务执行，每个任务完成后进行规范合规审查、代码质量审查，并更新 `task_plan.md`、`progress.md`、`findings.md`。

14.3 CodeReview 标准提示

```
/cx-aicode:cx-codereview --scope=spec --spec=MES-0535 --tech=auto
```

请只审查本次 SPEC 范围内的变更，不要把历史遗留问题混入本次结论。
报告必须写入 `.cx/reports/codereview/`，并更新 `.cx/status/index.json`。
L1/L2 问题不得直接通过。

14.4 Verify 标准提示

```
/cx-aicode:cx-verify
```

请基于 `design.md`、`tasks.md`、CodeReview 报告、测试/构建证据进行五维验证。
如果证据不足，请输出 `insufficient_evidence`，不要声称通过。

14.5 Bug 修复标准提示

```
/cx-aicode:cx-debug
```

问题现象：...
复现步骤：...
期望结果：...
实际结果：...
相关日志：...
最近变更：...

请先执行根因调查和假设验证，不要直接修改代码。
修复前先写能复现问题的失败测试，修复后运行回归测试。

14.6 处理 CodeReview 反馈提示

```
/cx-aicode:cx-feedback --from-codereview --to-task-plan
```

请将 CodeReview 报告中的待修复问题转换为 `.cx/work/task_plan.md` 的 CodeReview Fix Phase。
不要复制整份报告，只提取可执行修复任务，并在 `findings.md/progress.md` 中记录报告路径和处理策略。

15. 常见异常与处理

15.1 Hook 阻止写代码

可能原因：

- `.cx/` 未初始化。
- `design.md` 未批准。
- `.cx/work/cx-work-session.json` 不存在或无效。
- `task_plan.md` 中 SPEC-ID 与 session marker 不一致。
- 当前操作试图绕过 `cx-work` 写实现文件。

处理：

```
/cx-aicode:cx-status  
/cx-aicode:cx-doctor
```

然后检查：

```
.cx/specs/<SPEC-ID>/design.md  
.cx/specs/<SPEC-ID>/tasks.md  
.cx/work/cx-work-session.json  
.cx/work/task_plan.md
```

15.2 cx-work 找不到 SPEC-ID

处理：

```
/cx-aicode:cx-work --spec=<SPEC-ID>
```

如果仍失败，检查：

- `.cx/specs/<SPEC-ID>/design.md` 是否存在。
- `.cx/specs/<SPEC-ID>/tasks.md` 是否存在。
- `.cx/status/index.json.activeChange` 是否错误。

15.3 CodeReview 通过但 Verify 失败

这是合理情况。CodeReview 检查代码质量，Verify 检查需求符合性。

处理：

1. 读取 Verify 报告。
2. 将失败项转为任务。
3. 回到 `cx-work` 修复。
4. 重新运行 CodeReview 与 Verify。

15.4 Verify 输出 insufficient_evidence

原因：缺少新鲜测试/构建证据。

处理：

```
/cx-aicode:cx-test --type=all --lang=<tech>
```

或手工运行项目测试命令，例如：

```
mvn test
npm test
pytest
dotnet test
```

然后重新运行：

```
/cx-aicode:cx-verify
```

15.5 Agent 输出无法写入状态

可能原因：

- 缺少 `cx-agent-result` 代码块。
- `status`、`nextAction`、`riskLevel` 值不合法。
- `issues` 不是数组。
- `CodeReview passed` 但存在 L1/L2。
- 报告路径不在 `.cx/reports/<type>/` 下。

处理：

- 重新要求 Agent 按契约输出。
- 检查 `scripts/agent/validate-agent-result.js` 的输出。
- 检查 `scripts/agent/update-status.js` 的错误信息。

15.6 Windows PowerShell 执行问题

处理建议：

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
chcp 65001
```

并确认：

- PowerShell 5.1 可用。
- 脚本文件编码为 UTF-8 BOM + CRLF。
- 不使用 PowerShell 7 专属语法。

16. 安全与合规要求

16.1 代码安全

AI 编程过程中必须关注：

- 密钥、密码、Token 不得硬编码。
- SQL 不得拼接用户输入。
- 日志不得输出敏感信息。
- 权限校验不得遗漏。
- 文件路径不得允许目录穿越。
- 外部命令执行必须白名单化。
- 生产配置不得被测试配置覆盖。

16.2 数据安全

使用外部模型时，企业应额外加入脱敏机制：

- 公司名脱敏。
- 人名脱敏。
- 电话、邮箱、工号脱敏。
- 密码、Token、密钥过滤。
- 设备编号、产线编号、客户名按敏感等级处理。

16.3 Agent 边界

只读 Agent 不得：

- 写文件。
- 运行命令。
- 调用其他 Agent。
- 修改状态。
- 更新报告。
- 修改 `.cx/work/*`。

这类操作必须由主流程或脚本执行。

16.4 Git 操作边界

插件收尾流程不应主动执行：

- `git commit`

- git push
- git merge
- 创建远程 MR/PR

企业可在插件外部制定单独发布流程。

17. 交付物验收清单

17.1 需求设计阶段

☐

`.cx/specs/<SPEC-ID>/source.md` 已生成。

☐

`metadata.yaml` 已记录需求类型和源文件。

☐

`design.md` 包含接口、数据模型、业务规则、异常处理、测试策略。

☐

`tasks.md` 任务可执行、可验收。

☐

`design.md` 包含 `## Approval` 和 `**APPROVED**`。

17.2 编码执行阶段

☐

`.cx/work/cx-work-session.json` 有效。

☐

`task_plan.md` 绑定正确 SPEC-ID。

☐

每个任务均经过实现、规范合规审查、代码质量审查。

☐

每个任务完成后 checkbox 已更新。

☐

`progress.md` 记录任务、测试和变更文件。

☐

`findings.md` 记录关键发现和决策。

17.3 CodeReview 阶段

☐

报告写入 `.cx/reports/codereview/`。

☐

`.cx/status/index.json.reviewStatus` 已更新。

☐

`L1=0`。

☐

`L2=0`，或有明确 `conditional_approved` 审批证据。

☐

修复任务已接入 `task_plan.md` 或记录为技术债。

17.4 Verify 阶段

☐

有新鲜测试/构建证据。

☐

Verify 报告写入 `.cx/reports/verify/`。

☐

`verifyStatus=passed`。

☐

五维验证均有证据。

17.5 收尾阶段

☐

`cx-finish` 已检查 Review 和 Verify 状态。

☐

测试最终通过。

☐

决定保留或丢弃工作。

☐

未由插件擅自 `commit/push/merge`。

☐

归档需求与报告。

☐

记录经验教训。

18. 命令速查表

18.1 标准流程命令

阶段	命令
初始化	/cx-aicode:cx-init --flavor=mes
状态检查	/cx-aicode:cx-status
诊断	/cx-aicode:cx-doctor
需求探索	/cx-aicode:cx-brainstorm --spec-id <ID> --source <file>
规范定稿	/cx-aicode:cx-spec new --spec-id <ID> --source <file>
执行任务	/cx-aicode:cx-work --spec=<ID>
代码审查	/cx-aicode:cx-codereview --scope=spec --spec=<ID> --tech=auto
反馈处理	/cx-aicode:cx-feedback --from-codereview --to-task-plan
设计验证	/cx-aicode:cx-verify
收尾	/cx-aicode:cx-finish
归档	/cx-aicode:cx-archive

18.2 辅助命令

场景	命令
代码分析	/cx-aicode:cx-analyze --tech=auto
生成测试	/cx-aicode:cx-test --type=all --lang=<tech>
QA 计划	/cx-aicode:cx-qa --plan --feature=<功能名> --write
文档生成	/cx-aicode:cx-doc
图表生成	/cx-aicode:cx-diagram
文档合规	/cx-aicode:cx-doc-check --doc=<待检查文档> --template=<模板名或模板路径>
Bug 调试	/cx-aicode:cx-debug
TDD	/cx-aicode:cx-tdd
项目复制	/cx-aicode:cx-project-copy --ref=<参考项目路径> --new=<新项目路径>
经验记录	/cx-aicode:cx-learn --type <类型> "问题描述 / 解决方案"
插件升级	/cx-aicode:cx-update --check

19. 企业落地建议

19.1 试点阶段

建议选择一个中等规模、风险可控的需求作为试点：

- 有明确需求文档。
- 代码范围不超过 5-10 个核心文件。
- 有现成测试或可补充测试。
- 技术栈属于插件支持范围。

试点目标不是“让 AI 一次写完”，而是验证：

- 设计批准门禁是否有效。
- 三文件机制是否可恢复。
- Agent-first 审查是否能发现问题。
- CodeReview 与 Verify 能否产生可靠报告。
- 开发人员是否能按 SOP 操作。

19.2 推广阶段

推广前应建立：

1. 项目级 `.cx/config.json` 模板。
2. 技术栈自定义 `code_rules.md` 和 `scoring_rules.yml`。
3. 审批人和 conditional approval 规则。
4. QA 证据格式。
5. Git 分支和发布流程。
6. 脱敏与模型调用策略。
7. 插件升级和回滚策略。

19.3 度量指标

建议统计：

- 每个需求从 Brainstorm 到 Finish 的耗时。
- 每个任务 AI 一次通过率。
- CodeReview L1/L2/L3/L4 数量。
- Verify 失败原因分布。
- 测试新增数量和通过率。
- 人工介入次数。
- 返工次数。
- 需求遗漏率。

20. 最终结论

基于本次代码分析，`cx-aicode v2.3.6` 已具备较完整的企业级 AI 编程流程治理能力。其价值不在于“多几个命令”，而在于把 AI 编程过程拆成可审计、可验证、可恢复的工程流程：

- `cx-brainstorm/cx-spec` 控制需求和设计入口。
- `cx-work` 控制逐任务实现和两阶段审查。
- `cx-codereview` 控制代码质量门禁。
- `cx-verify` 控制需求符合性门禁。
- `cx-finish` 控制本地收尾边界。
- `Hooks` 和 `.cx/status/index.json` 控制跨会话状态和准入门禁。
- 只读 Agent 与报告脚本形成职责分离。

企业内部使用时，最重要的是严格执行以下五条底线：

1. **设计未批准，不写实现代码。**
2. **每个任务必须经过规范合规审查和代码质量审查。**
3. **CodeReview 未通过，不进入 Verify。**
4. **Verify 无新鲜证据，不声明完成。**
5. **状态文件、报告和测试结果必须可追溯。**

只要团队按本 SOP 执行，插件可以从“AI 辅助写代码工具”升级为“企业级 AI 编程治理流程”的核心组件。